IF5100 — Exercise Set

Topic 1 — Python Fundamentals & Control Flow (1b)

1.1 (concept) Classify each as *declarative* or *imperative* knowledge: (a) "√x is the y such that y·y = x"; (b) a step-by-step bisection procedure to approximate √x; (c) "a prime has exactly two divisors."

1.2 (concept) For each snippet, say whether it's a *syntax error*, a *static semantic error*, or *runs fine*: (a) `3 + "hi"`; (b) `print("hello"`; (c) `3.2 * 5`; (d) `"5"5`.

1.3 (trace) Predict the output:

```
x, y = 7, 2
print(x / y, x // y, x % y, x ** y)
```

1.4 (trace) Predict the output, and note each value's type:

```
print(int(3.9))
print(float(3))
print(int("3") + 4)
print(3 * "2")
```

1.5 (trace) What does `area` print, and why?

```
pi = 3.14
radius = 2
area = pi * radius ** 2
radius = radius + 1
print(area)
```

1.6 (trace) The user types 2 at the prompt. What is printed?

```
text = input("Enter: ")
print(5 * text)
```

1.7 (concept) Evaluate each to `True`/`False`: (a) `3 < 5 and 5 < 4`; (b) `not (3 == 3)`; (c) `"apple" < "banana"`; (d) `"Zebra" < "apple"`. Explain (d).

1.8 (trace) Give the output:

```
n, total = 0, 0
while n < 5:
    total += n
    n += 1
print(total)
```

1.9 (write) Write a `for` loop using `range` that prints every multiple of 3 from 3 to 30 inclusive.

1.10 (write) Write **FizzBuzz** for 1–20: print `Fizz` for multiples of 3, `Buzz` for multiples of 5, `FizzBuzz` for both, else the number.

1.11 (concept) The 3b deck ends by asking "Is using `break` good practice?" Give one argument for and one against, with a scenario for each.

Topic 2 — Abstraction & Functions (1c)

2.1 (concept) In your own words, distinguish *abstraction* from *decomposition*, and name the Python mechanism each is achieved with in this week's material.

2.2 (trace) What is printed, and why is one line `None`?

```
def greet(name):
    print("Hi " + name)

result = greet("Ana")
print(result)
```

2.3 (concept) Explain the scope rule: inside a function you *can* read an outer variable but *cannot* rebind it (without `global`). Why is `global` "frowned upon"?

2.4 (trace) Give the output:

```
x = 3
def f(y):
    x = y + 1
    return x
print(f(10))
print(x)
```

2.5 (write + docstring) Write `is_leap_year(year)` with a PEP 257 docstring. Rule: divisible by 4, except centuries unless divisible by 400.

2.6 (write, functions as arguments) Write `apply_twice(func, x)` that returns `func(func(x))`. Show it called with a `lambda`/function that increments by 1, for `x = 5`.

2.7 (fix the bug) This should return the larger of two numbers but misbehaves. Find and fix the bug:

```
def bigger(a, b):
    if a > b:
        result = a
    return result
```

Topic 3 — Modules & Files (2a)

3.1 (concept) After `import circle`, why do `pi` and `circle.pi` refer to different objects? What does `from circle import *` change about that?

3.2 (concept) Give one concrete reason to use `import numpy as np` (the `as` form) rather than plain `import numpy`.

3.3 (trace) Given `circle.py` (with `pi = 3.14159`, `area(r)` = `pi*r**2`), what prints?

```
import circle
pi = 3
print(pi)
print(circle.pi)
print(circle.area(2))
```

3.4 (write, finger exercise from deck) Write a program that writes the first ten positive integers to `intpos_file.txt`, one per line, then reopens the file and prints them.

3.5 (concept) Rewrite this using a `with` statement and explain what problem `with` prevents:

```
f = open("data.txt", "w")
f.write("hello")
f.close()
```

3.6 (fix the bug) A student wants to *append* a line to an existing log but keeps overwriting it. What's wrong?

```
with open("log.txt", "w") as f:
    f.write("new entry\n")
```

Topic 4 — Structured Types & Mutability (2b)

4.1 (trace, slicing) With `s = "abcdefgh"`, evaluate: (a) `s[3:6]`; (b) `s[::-1]`; (c) `s[4:1:-2]`; (d) `s[-2]`.

4.2 (concept) Why does `s[0] = "y"` raise an error for a string, while `s = "y" + s[1:]` is allowed?

4.3 (concept) Classify each as *mutable* or *immutable*: `int`, `str`, `tuple`, `list`, `dict`.

4.4 (trace, the classic aliasing trap) Predict both lists:

```
warm = ["red", "yellow"]
hot = warm
hot.append("orange")
print(warm)
print(hot)
```

Then change `hot = warm` to `hot = warm[:]` and give the new output.

4.5 (trace, mutation during iteration — from the deck) Why does L1 end as `[2, 3, 4]` rather than `[3, 4]`?

```
def remove_dups(L1, L2):
    for e in L1:
        if e in L2:
            L1.remove(e)

L1 = [1, 2, 3, 4]
L2 = [1, 2, 5, 6]
remove_dups(L1, L2)
print(L1)
```

4.6 (write, dict) Write `lyrics_to_frequencies(words)` that takes a list of words and returns a dict mapping each word to its count.

4.7 (write, dict + tuple) Using the frequency dict, write `most_common(freqs)` returning a tuple `(list_of_words, count)` for the word(s) with the highest count.

4.8 (concept) Why can a `tuple` be a dictionary key but a `list` cannot?

Topic 5 — Recursion (3a)

5.1 (concept) Every correct recursion needs two things. Name them, and say what goes wrong if the second is missing.

5.2 (trace) Draw the call stack / give the return value:

```
def fact(n):
    if n == 1:
        return 1
    return n * fact(n - 1)
print(fact(4))
```

5.3 (write) Write recursive `mult(a, b)` (repeated addition) with base case `b == 1`, matching the deck's version.

5.4 (write) Write a recursive `is_palindrome(s)`. Assume `s` is already lowercased letters only. Base case: length 0 or 1.

5.5 (write) Write recursive `fib(n)` with base cases `fib(0)`=1, `fib(1)`=1.

5.6 (concept + write) Naive `fib(5)` makes 15 calls. Explain *why* that's wasteful, then rewrite `fib` with **memoization** (a cache dict).

5.7 (write, harder) Write `hanoi(n, source, target, spare)` that prints the sequence of moves to shift n disks. How many moves for `n = 3`?

5.8 (concept) State the induction argument for why `mult(a, b)` returns the correct answer for all `b ≥ 1`.

Topic 6 — Exceptions & Assertions (3b)

6.1 (concept) Name the exception each raises: (a) `[1,7,4][4]`; (b) `int("hello")`; (c) `undefined_var`; (d) `"a" / 4`; (e) `1 / 0`.

6.2 (write) Wrap two `int(input(...))` reads and a division in `try`/`except` with *separate* handlers for `ValueError` and `ZeroDivisionError`, plus a catch-all.

6.3 (concept) The deck lists three ways to handle a bad case in `avg(grades)`: print a warning, return `0.0`, or re-`raise`. Give a situation where each is the right choice.

6.4 (trace) What prints, in what order?

```
try:
    print("A")
    x = 1 / 0
    print("B")
except ZeroDivisionError:
```

```
    print("C")
else:
    print("D")
finally:
    print("E")
```

6.5 (write, assertion) Rewrite `avg(grades)` to use an `assert` guaranteeing a non-empty list, with a message.

6.6 (concept) When should you use an **assertion** versus raising/handling an **exception**? (Hint: programmer error vs. expected bad user input.)

Topic 7 — Simple Numerical Programs (3c)

7.1 (concept) Define *exhaustive enumeration* and give one strength and one weakness.

7.2 (write) Write `is_prime(x)` by testing divisors from 2 to x−1 (the deck's simplest method). Then state one optimization (hint: √x).

7.3 (write) Write `sqrt_bisection(x, epsilon)` that finds √x by bisection to within `epsilon`. Roughly how does its step count compare with exhaustive enumeration?

7.4 (concept) Newton–Raphson converges faster than bisection. In one sentence, why — and where in machine learning does this idea reappear (per the deck)?

7.5 (write, links to Topic 5) Convert your bisection `sqrt` into a **recursive** version. Why might it need a *helper function* to make the first call?

Topic 8 — Object-Oriented Programming (4a)

8.1 (concept) An object bundles three things (per the deck). Name them, using `[1,5,7]` (a `list`) as your example.

8.2 (write) Implement the `Coordinate` class from the deck: `__init__(self, x, y)`, a `distance(self, other)` method, and a `__str__` that prints <x,y>.

8.3 (trace) Given that `Coordinate`, what prints?

```
c = Coordinate(3, 4)
zero = Coordinate(0, 0)
print(c.distance(zero))
print(c)
```

8.4 (concept) Explain *information hiding*: why should code outside the class call `a.get_age()` instead of `a.age`? What breaks if the author renames `age` to `years`?

8.5 (write, the deck's Fractions exercise) Create a `Fraction` class storing numerator and denominator as ints, supporting `__add__`, `__str__`, an `invert()` method, and a `to_float()`.

8.6 (write, inheritance) Given a `Mammal` base class with `age`, `name`, getters/setters and `__str__`, write a `Cat(Mammal)` subclass that adds `speak()` (prints "meow") and overrides `__str__`. Note which methods `Cat` inherits without rewriting.

8.7 (write, class variable) Implement `Rabbit(Mammal)` using a class variable `tag` to assign each new rabbit a unique id. Explain why `tag` is shared across all instances.

8.8 (write, operator overloading) Add `__add__` to `Rabbit` so `r1 + r2` returns a new age-0 `Rabbit` whose parents are `r1` and `r2`, and `__eq__` so two rabbits are equal iff they share the same two parents.

8.9 (concept) When looking up a method on an instance, how does Python decide which class's version to use? Illustrate with `Cat`, `Mammal`, `object`.